Tài liệu thuyết minh sản phẩm

R2AI 2026 · Stage 3 · ViBioMIR — đội loi_cuda

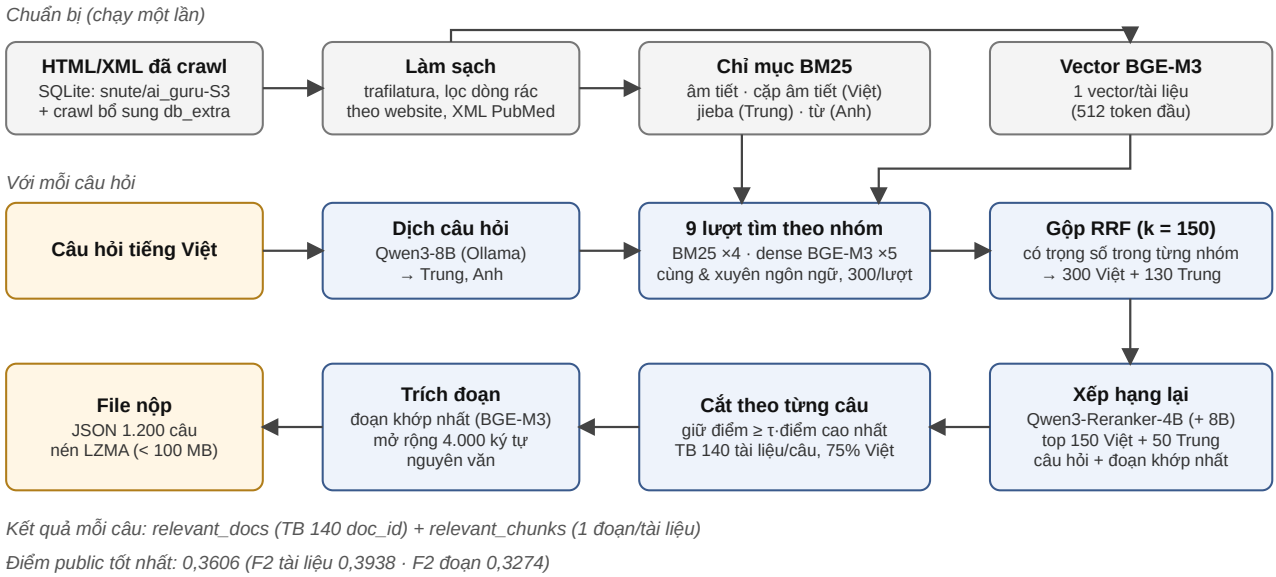
Mã nguồn: https://github.com/sonIT2005/R2AI-retrieval-of-medical-documents · Dữ liệu: https://drive.google.com/drive/folders/1zoATII9wMD3SAhux7n9Cu49K1jRjEwYQ?usp=sharing

1.1. Tổng quan hệ thống

Bài toán. Với mỗi câu hỏi y khoa tiếng Việt, trả về danh sách tài liệu liên quan (doc_id trong danh sách 1.220.598 URL của BTC: Việt, Trung, PubMed tiếng Anh) và một đoạn nội dung trích nguyên văn cho mỗi tài liệu. Đánh giá bằng F2 cấp tài liệu và F2 cấp đoạn.

Phương pháp. Hệ thống truy hồi lai hai tầng, chạy hoàn toàn trên một máy (GPU 16 GB):

1. **Truy hồi tầng một** — chín lượt tìm riêng theo từng nhóm ngôn ngữ (xác định theo website): BM25 theo âm tiết và theo cặp âm tiết liền nhau cho tiếng Việt, BM25 tách từ jieba cho tiếng Trung, BM25 cho tiếng Anh, và năm lượt dense BGE-M3 (câu Việt → tài liệu Việt / Trung / Anh; câu dịch Trung → tài liệu Trung; câu dịch Anh → tài liệu Anh).
2. **Gộp thứ hạng** — Reciprocal Rank Fusion có trọng số trong từng nhóm, k = 150; giữ 300 ứng viên Việt và 130 ứng viên Trung mỗi câu (nhóm Anh có hạn mức 0: bài nộp thử chỉ gồm tài liệu PubMed đạt 0 điểm).
3. **Xếp hạng lại** — Qwen3-Reranker chấm 150 ứng viên Việt và 50 ứng viên Trung đầu tiên; đầu vào là câu hỏi đầy đủ cùng tiêu đề và đoạn khớp nhất của tài liệu; điểm là xác suất token “yes”. Bài nộp cuối dùng trung bình điểm của bản 4B và 8B.
4. **Chọn kết quả đầu ra** — số tài liệu thay đổi theo từng câu: giữ tài liệu có điểm ≥ τ·(điểm cao nhất của câu đó), τ được dò để trung bình 140 tài liệu/câu (75% Việt); mỗi tài liệu kèm một đoạn 4.000 ký tự (Trung: 2.000) quanh vị trí khớp nhất.



Hình 1. Luồng xử lý từ câu hỏi đầu vào đến danh sách tài liệu và đoạn nội dung.

Chi tiết từng thành phần

Thành phần	Cách làm	Mã nguồn
Chỉ mục BM25	Vector hoá bằng hashing (2^{23} chiều; cặp âm tiết 2^{24}), k1 = 1,2, b = 0,75, 10.000 ký tự đầu của tiêu đề + nội dung; ma trận trọng số BM25 lưu sẵn ở `index/bm25/`	search_docs.py
Chỉ mục dense	BGE-M3 fp16, 512 token đầu tài liệu, chuẩn hoá L2; tìm bằng tích vô hướng trên GPU	embed_corpus.py, search_docs.py
Gộp RRF	$s(d) = \Sigma\ w_{\text{lượt}} / (150 + \text{hạng})$; trọng số: BM25 Việt 0,33 · cặp âm tiết 0,2 · BM25 Trung 0,3 · dense 1	search_docs.py
Đoạn khớp nhất	Chia tài liệu thành đoạn ~1.000 ký tự (Trung ~500), giữ tiêu đề mục với đoạn sau; chọn đoạn có max(cos với câu Việt, cos với bản dịch cùng ngôn ngữ) theo BGE-M3	make_submission.py
Xếp hạng lại	Qwen3-Reranker-4B fp16 (8B nạp 8-bit), lô 12.000 token; điểm lưu đệm theo (câu, tài liệu)	qwen_rerank.py, ensemble_rerank.py
Cắt theo câu	τ riêng cho Việt và Trung, dò nhị phân; mỗi câu kẹp trong [80, 400] tài liệu	dyn_cut.py
File nộp	JSON 1.200 câu, tự kiểm tra định dạng và tính nguyên văn của đoạn, nén LZMA (từ điển = cỡ file)	make_submission.py

Kết quả trên bảng xếp hạng public (600 câu)

Cấu hình	F2 tài liệu	F2 đoạn	Điểm
RRF 8 lượt, 100 tài liệu/câu, đoạn ~1.000 ký tự (bản đầu)	0,2375	0,0837	0,1606
+ đoạn 4.000 ký tự, bỏ nhóm Anh, giảm trọng số BM25 Việt	0,3038	0,2434	0,2736
+ cắt số tài liệu theo từng câu, đoạn cho mọi tài liệu	0,3467	0,2919	0,3193
+ RRF k = 150, 75% Việt, BM25 cặp âm tiết	0,3748	0,3135	0,3442
+ Qwen3-Reranker-4B, cắt theo điểm reranker, TB 140 tài liệu/câu	0,3938	0,3274	**0,3606**

1.2. Dữ liệu

Nguồn và thu thập

Dữ liệu	Nguồn	Quy mô
Danh sách kho + câu hỏi	BTC: Hugging Face `AIGuruTinix/ViBioMIR` (links_corpus_vi_en_zh.parquet, query.parquet)	1.220.598 URL; 1.200 câu
Nội dung đã crawl (dùng chung)	Hugging Face `snute/ai_guru-S3`: db/ (97 website), db_cc/ (Common Crawl), db_wayback/ (Wayback Machine), db_alt/ — SQLite, HTML nén zstd	~58 GB
Crawl bổ sung của đội	`vibiomir_s3/db_extra/`: các URL còn thiếu, tải bằng curl_cffi, kiểm tra nội dung (loại trang CAPTCHA trả mã 200) — scripts/crawl/	38 file SQLite, 6,1 GB
Bản dịch câu hỏi	Qwen3-8B, có trong repo: results/queries_translated.jsonl	1.200 câu
Danh sách ứng viên + điểm reranker	Có trong repo: results/D_bi02.jsonl, results/qwen_rerank_cache_q3r4B_passage.pkl, results/qwen_rerank_cache_q3r8B_passage.pkl → tái tạo bài nộp không cần chạy lại tìm kiếm và reranker	~22 MB

Làm sạch và phân đoạn

- Giải mã theo bảng mã khai báo (GBK/GB2312 → gb18030); trích nội dung chính bằng trafilatura (favor_recall, giữ bảng).
- Lấy mẫu 1.500 trang mỗi website, coi dòng xuất hiện ở ≥ 3% số trang là rác giao diện và loại bỏ; trang quá ngắn sau trích xuất dùng toàn bộ chữ trong thân trang (bỏ script, nav, header, footer).
- PubMed (XML efetch): tiêu đề, tóm tắt theo mục, từ khoá, MeSH. Chỉ giữ id thuộc danh sách BTC.
- Kết quả: 1.205.706 tài liệu (98,8% danh sách): 729.490 Việt, 236.518 Trung, 239.698 PubMed → `clean/corpus/*.parquet` (doc_id, url, host, title, text, lang).
- Phân đoạn khi tạo file nộp: gom dòng tới ~1.000 ký tự (Trung ~500), mỗi đoạn là lát cắt nguyên văn của văn bản.

Hướng dẫn truy cập

Google Drive: https://drive.google.com/drive/folders/1zoATII9wMD3SAhux7n9Cu49K1jRjEwYQ?usp=sharing — giải nén từ thư mục gốc repo bằng `tar -xf <file>.tar`, kiểm tra bằng `sha256sum -c SHA256SUMS`.

File	Nội dung	Bắt buộc?
vibiomir_db_extra.tar	vibiomir_s3/db_extra/ — crawl bổ sung của đội (6,1 GB)	Có
vibiomir_clean_corpus.tar	clean/corpus/ — văn bản đã làm sạch (1,5 GB)	Không (thay bước làm sạch)
vibiomir_emb.tar	emb/ — vector BGE-M3 (2,4 GB)	Không (thay bước tạo vector)
vibiomir_results.tar	results/: 9 lượt tìm, vector đoạn BGE-M3 (~3,6 GB)	Không (thay ~1 giờ GPU)

Dữ liệu dùng chung tải bằng: `hf download snute/ai_guru-S3 --repo-type dataset --local-dir vibiomir_s3` và `hf download AIGuruTinix/ViBioMIR links_corpus_vi_en_zh.parquet query.parquet --repo-type dataset --local-dir btc_data_v2`.

1.3. Mô hình sử dụng

Tất cả là mô hình mã nguồn mở ≤ 15 tỷ tham số, phát hành trước 01/08/2026; không dùng API thương mại. Mô hình Hugging Face tự tải khi chạy lần đầu (thư mục mặc định `~/.cache/huggingface/hub`).

Mô hình	Vai trò	Phiên bản	Đường dẫn tải chính thức
BAAI/bge-m3 (568M)	Vector tài liệu, câu hỏi; chọn đoạn khớp nhất	commit 5617a9f	huggingface.co/BAAI/bge-m3
Qwen/Qwen3-Reranker-4B	Xếp hạng lại (fp16)	commit 22e6836	huggingface.co/Qwen/Qwen3-Reranker-4B
Qwen/Qwen3-Reranker-8B	Xếp hạng lại, gộp với 4B (8-bit, bitsandbytes)	commit 77d193c	huggingface.co/Qwen/Qwen3-Reranker-8B
Qwen3-8B (Ollama qwen3:8b, Q4_K_M)	Dịch câu hỏi sang Trung, Anh	digest 500a1f067a9f	`ollama pull qwen3:8b`

Cấu hình và prompt. Dịch câu hỏi: nhiệt độ 0, tắt chế độ suy nghĩ, đầu ra ràng buộc JSON schema {"zh", "en"}; system prompt:

```
You are a professional medical translator. Translate the Vietnamese medical question into: 1.
zh: Simplified Chinese, phrased naturally like a patient asking on a Chinese medical Q&A
website, using standard Chinese medical terms. 2. en: English, using standard medical
terminology as used in PubMed articles. Keep every number, unit, lab value, drug name and
abbreviation exactly as written. Translate faithfully: keep all details, add nothing, and do not
answer the question.
```

Xếp hạng lại (định dạng chuẩn của Qwen3-Reranker; điểm = P(“yes”) / (P(“yes”) + P(“no”)) ở token cuối):

```
<|im_start|>system
Judge whether the Document meets the requirements based on the Query and the Instruct provided.
Note that the answer can only be "yes" or "no".<|im_end|>
<|im_start|>user
<Instruct>: Given a Vietnamese patient's medical question, judge whether the document is
relevant to it
<Query>: {câu hỏi}
<Document>: {tiêu đề}
{đoạn khớp nhất ≤ 1.200 ký tự}<|im_end|>
<|im_start|>assistant
<think>

</think>
```

Chỉ dùng khi phát triển (không nằm trong quy trình tạo bài nộp): Qwen3-14B và Gemma 3 12B (Ollama) làm giám khảo cho một tập phát triển tự gán nhãn trên 300 câu (`llm_judge.py`), dùng để so sánh các phương án xếp hạng lại.

1.4. Mã nguồn

Toàn bộ mã nguồn: https://github.com/sonIT2005/R2AI-retrieval-of-medical-documents.

File	Chức năng
clean_corpus.py	Làm sạch HTML/XML → clean/corpus
embed_corpus.py	Vector BGE-M3 → emb/
translate_queries.py	Dịch câu hỏi (Qwen3-8B qua Ollama)
search_docs.py	Chỉ mục BM25, 9 lượt tìm, gộp RRF
qwen_rerank.py · ensemble_rerank.py	Xếp hạng lại bằng Qwen3-Reranker; gộp điểm 4B + 8B
dyn_cut.py	Cắt số tài liệu theo từng câu
make_submission.py	Phân đoạn, chọn đoạn, kiểm tra định dạng, nén
build_submission.sh	Bước gộp → xếp hạng lại → cắt → tạo file nộp trong một lệnh (tham số = cấu hình bài nộp)
scripts/crawl/	Crawl bổ sung (find_missing.py, crawl_missing2.py, …)
tools/pack_data.sh	Đóng gói dữ liệu chia sẻ

Môi trường	Thư viện chính (phiên bản)
conda base, Python 3.14 — làm sạch	trafilatura 2.3.1 · zstandard 0.25.0 · lxml 6.1.3 · curl_cffi 0.16.3 · pyarrow 25.0.1 · pandas 3.0.6 (requirements-clean.txt)
conda vibiomir, Python 3.11 — GPU	torch 2.11.0+cu128 · sentence-transformers 6.1.0 · transformers 5.19.0 · bitsandbytes 0.50.2 · accelerate 1.15.0 · scikit-learn 1.9.1 · jieba 0.42.1 · pyarrow 26.0.0 (requirements-search.txt)
Dịch vụ	Ollama 0.31.1 · NVIDIA driver 595.91, CUDA 12.8

Cấu hình tái hiện bài nộp (tham số mặc định trong build_submission.sh): RRF k = 150; hạn mức 300 Việt / 130 Trung / 0 Anh; reranker chấm 150 Việt + 50 Trung, đoạn ≤ 1.200 ký tự; cắt trung bình 140 tài liệu/câu, 75% Việt, kẹp [80, 400]; đoạn nộp 4.000 ký tự, nén LZMA-XL.

1.5. Hướng dẫn cài đặt và chạy lại

Chi tiết đầy đủ trong `README.md` của repo. Tóm tắt:

```
git clone https://github.com/sonIT2005/R2AI-retrieval-of-medical-documents.git vibiomir && cd vibiomir
pip install -r requirements-clean.txt                          # conda base, Python 3.14
conda create -n vibiomir python=3.11 -y && conda activate vibiomir
pip install torch==2.11.0 --index-url https://download.pytorch.org/whl/cu128
pip install -r requirements-search.txt && ollama pull qwen3:8b
# dữ liệu: hf download (mục 1.2) + tar -xf vibiomir_db_extra.tar
python clean_corpus.py                                         # ~1 giờ 20 (conda base)
python embed_corpus.py                                         # ~3 giờ GPU
python search_docs.py --quota 300,130,0 --rrf-k 150 --out D_bi02   # ~20 phút
./build_submission.sh prr_m140                                 # Qwen3-Reranker-4B → bài 0,3606
RERANK=ens ./build_submission.sh ens_m140                      # gộp 4B + 8B → bài vòng private
```

Tái tạo nhanh từ dữ liệu tăng tốc (không cần làm sạch, tạo vector, chạy reranker):

```
CAND=results/D_bi02.jsonl ./build_submission.sh prr_m140
CAND=results/D_bi02.jsonl RERANK=ens ./build_submission.sh ens_m140
```